

PMDM06.- Multimedia. Audio y Vídeo.



[Ministerio de Educación y Formación Profesional](https://eadfp.murciaeduca.es/pluginfile.php/116996/mod_resource/content/3/singlepage_index.html). (Dominio público)

Materiales formativos de FP Online propiedad del Ministerio de Educación y Formación Profesional.

[Aviso Legal](#) 

Caso práctico

- Equipo, tenemos una petición de un cliente - anuncia Ada-. Tenemos que realizar una audioguía.

Ada les informa de que uno de los clientes de **BK Programación** ha solicitado presupuesto para desarrollar una app cuya funcionalidad principal es ser una audioguía para uno de los principales monumentos de la ciudad, muy visitado tras haber sido restaurado. La ventaja de construir esta app es que no es necesario comprar tantos dispositivos como visitantes concurrentes existan, sino que es posible contar con un parque de dispositivos, y para los días de mayor afluencia de público, los usuarios que contraten este servicio simplemente pueden descargar la app en sus propios teléfonos móviles para acceder a la información sobre el monumento.



[Rahul Pandit \(Pexels\)](#)

- De modo que el parque de dispositivos tendrá guardado en local los medios a reproducir -apunta Juan, muy atento a la explicación del proyecto que les ha realizado Ada-, sin embargo, será necesario disponer de los medios a través de servicios de streaming y reproducción online para los usuarios que contraten el servicio.

- ¡Bien visto! -exclama María-. De otro modo ocuparían mucho espacio en los terminales de los clientes.

- ¿Nuevos controles a la vista? -pregunta Pedro.

- Para este proyecto es necesario integrar este tipo de medios en las aplicaciones Android Studio -explica Ada-, incorporando **Audio**, **Vídeo** o usando controles de tipo **WebView** donde incluso podemos seleccionar si habilitamos o no la capacidad de ejecutar código **JavaScript**.

1.- Audio.

Caso práctico



[Mikhail](#) ([Pexels](#))

- El primer elemento multimedia a incorporar serán los archivos de Audio -explica Ada-, que podrán ser reproducidos a través de la clase Media Player.
- ¿Qué formatos de audio es capaz de reproducir Android? -pregunta María.
- Android es capaz de reproducir distintos formatos -responde Ada-, pero uno de los más extendidos es el **mp3**.

El equipo de trabajo, gracias a algunos programas de **SW libre** y gratuitos como **Audacity**, puede generar los archivos que van a reproducirse en la app.

Para trabajar con la reproducción de sonido, existen en Android varias alternativas. Una de las más sencillas es trabajar con la clase **MediaPlayer**. Dicha clase permitirá crear un objeto **MediaPlayer** mediante el método **create()**, el cual recibirá como parámetro el propio sonido y que podrá encontrarse en la carpeta de recursos especial **raw**. Posteriormente reproduciremos dicho sonido con el método **start()**.

La clase **MediaPlayer** también se usará en la reproducción de vídeo.

Por otro lado, al igual que sucederá con los recursos de vídeo, es importante guardar los ficheros de sonido en una carpeta específica llamada **raw** dentro de la carpeta **res**.

1.1.- Ejercicio resuelto 1.

Ejercicio Resuelto

Vamos a crear una pequeña aplicación que permita reproducir el audio de la locución de unos números. La apariencia será la de la figura 1.

Al clicar en el texto de cualquiera de los números, aparecerá el texto en formato numérico y además se escuchará una locución relativa a ese número. Ver figura 2.

Figura 1



[Google Developers](#) (Uso educativo nc)

Figura 2



[Google Developers](#) (Uso educativo nc)

Mostrar retroalimentación

Lo primero será crear una carpeta de recursos (dentro de **res**) que llamaremos **raw** y que tendrá los ficheros de audio (en nuestro caso los meteremos en formato .mp3, pero reconoce otros formatos).

Una vez creada la carpeta, metemos los ficheros:

Para hacer el ejercicio el alumno debe tratar de crear dichos ficheros por cuenta propia. Se puede grabar un fichero con todos los números, y luego trocearlo con programas de tratamiento de sonido como Audacity.

Y codificamos lo siguiente:

activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/a
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    tools:context="com.example.dell.sonidomediaplayer.MainActivity"

    <TableLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content">
        <TableRow>
            <TextView
                android:id="@+id/tv1"
                android:layout_height="wrap_content"
                android:layout_weight="1"
                android:textSize="50dp"
                android:text="Uno"
                android:gravity="center"/>
            <TextView />

            <TextView
                android:id="@+id/tv2"
                android:layout_height="wrap_content"
                android:layout_weight="1"
                android:textSize="50dp"
                android:text="Dos"
                android:gravity="center"/>
            <TextView />
        </TableRow>
        <TableRow>
            <TextView
                android:id="@+id/tv3"
                android:layout_height="wrap_content"
                android:layout_weight="1"
                android:textSize="50dp"
```

```

39         android:text="Tres"
40         android:gravity="center"/>
41     <TextView />
42     <TextView
43         android:id="@+id/tv4"
44         android:layout_height="wrap_content"
45         android:layout_weight="1"
46         android:textSize="50dp"
47         android:text="Cuatro"
48         android:gravity="center"/>
49     <TextView />
50 </TableRow>
51 </TableLayout>
52 <TextView
53     android:id="@+id/tvNumero"
54     android:layout_width="wrap_content"
55     android:layout_height="wrap_content"
56     android:textSize="200dp"
57     android:layout_gravity="center"
58     android:textColor="@color/colorAccent"/>
</LinearLayout>

```

MainActivity.java

```

package com.example.dell.sonidomediaplayer;

import android.media.MediaPlayer;
import androidx.appcompat.app.AppCompatActivity;
import android.os.Bundle;
import android.view.View;
import android.widget.TextView;

public class MainActivity extends AppCompatActivity implements Vi

    TextView tv1, tv2, tv3, tv4, tvNumero;
    MediaPlayer mp;

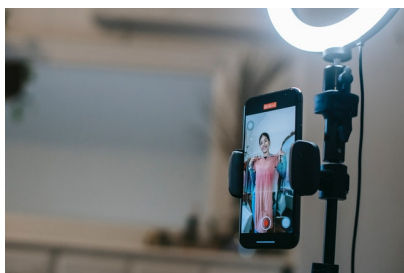
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        tv1=(TextView)findViewById(R.id.tv1);
        tv2=(TextView)findViewById(R.id.tv2);
        tv3=(TextView)findViewById(R.id.tv3);
        tv4=(TextView)findViewById(R.id.tv4);
        tvNumero=(TextView)findViewById(R.id.tvNumero);
        tv1.setOnClickListener(this);
        tv2.setOnClickListener(this);
        tv3.setOnClickListener(this);
        tv4.setOnClickListener(this);
    }

```

```
29     @Override
30
31     public void onClick(View v) {
32         switch(v.getId()){
33             case(R.id.tv1):
34                 tvNumero.setText("1");
35                 mp=MediaPlayer.create(this, R.raw.uno);
36                 break;
37             case(R.id.tv2):
38                 tvNumero.setText("2");
39                 mp=MediaPlayer.create(this, R.raw.dos);
40                 break;
41             case(R.id.tv3):
42                 tvNumero.setText("3");
43                 mp=MediaPlayer.create(this, R.raw.tres);
44                 break;
45             case(R.id.tv4):
46                 tvNumero.setText("4");
47                 mp=MediaPlayer.create(this, R.raw.cuatro);
48                 break;
49         }
50         mp.start();
51     }
52 }
```

2.- Vídeo.

Caso práctico



[Liza Summer \(Pexels\)](#)

- El siguiente elemento a incorporar en nuestra app es el vídeo -informa Ada a su equipo, que va cogiendo confianza con el paso de los días y la ejecución de sus proyectos-. Este medio puede generarse directamente grabando desde los dispositivos móviles actuales, aunque es imprescindible llevar a cabo la renderización y el procesamiento de los mismos para bajar el peso de los ficheros generados.

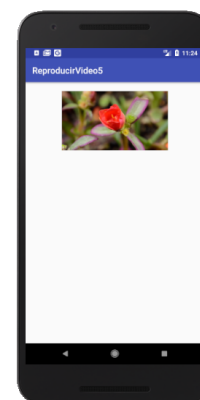
- ¿Tenemos también programas que nos ayuden a generar los vídeos? -pregunta Pedro.

- Por supuesto -responde Ada-. Existen distintos programas de SW libre que permiten llevar a cabo estas tareas como **Handbrake**.

- Otras alternativas son conseguir vídeos de bancos de imágenes y vídeos públicos -prosigue Ada en su respuesta-. En este caso es fundamental entender los términos en los que el autor comparte su obra y cede los derechos de su uso. Es importante cumplir con ellos para evitar problemas legales y para garantizar que los derechos de los autores se respetan, lo que garantizará el que estos se sigan generando en unos parámetros de calidad adecuados.

La clase más habitual para reproducir vídeo es la clase **VideoView**, y será ésta sobre la que nos centraremos. Además, la acompañaremos de la clase **MediaController** para proporcionar un reproductor o controlador de reproducción con controles de reproducción, pausa, avance y retroceso, con una apariencia sencilla pero correcta.

La clase **VideoView** muestra un archivo de vídeo. La clase **VideoView** puede cargar imágenes de varias fuentes (como recursos o proveedores de contenido), se ocupa de calcular su medición a partir del vídeo para que se pueda usar en cualquier administrador de diseño y proporciona varias opciones de visualización, como escalado y tintado.



[Ministerio de Educación y Formación Profesional \(Uso Educativo no\)](#)

VideoView no conserva su estado completo al pasar al segundo plano o al cambiar la orientación del dispositivo. En particular, no restaura el estado de reproducción actual, posición de reproducción, pistas seleccionadas o cualquier pista de subtítulos agregada a través de **addSubtitleSource()**. Las aplicaciones deben guardarlas y restaurarlas por sí mismas en **Activity.onSaveInstanceState(Bundle)** y

Activity.onRestoreInstanceState(Bundle). En el último ejercicio resuelto del capítulo trabajaremos sobre este importante aspecto.

Existen otras clases y librerías para reproducción de vídeo, como **ExoPlayer**, pero se puede considerar **VideoView** como el referente principal.

En los ejercicios resueltos que proporcionaremos a continuación, los vídeos a reproducir unas veces pertenecerán a la aplicación (incorporados en carpeta **res/raw**, y otras veces serán vídeos de internet (servidor externo) o incluso, podrán ser vídeos que tengamos incorporados en un servidor web local (por ejemplo con el Apache que proporciona XAMPP).

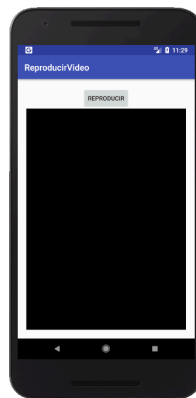
2.1.- Ejercicio Resuelto 2.

Ejercicio Resuelto

Crear una aplicación que disponga de un botón y un **VideoView** de forma que al pulsar al botón reproduzca, de manera aleatoria, dos posibles vídeos que previamente habremos incorporado en la aplicación (ver figura 1).

Los posibles vídeos serán los mostrados en la figura 2 y figura 3.

Figura 1



[Google Developers](#) (Uso educativo nc)

Figura 2



[Google Developers](#) (Uso educativo nc)

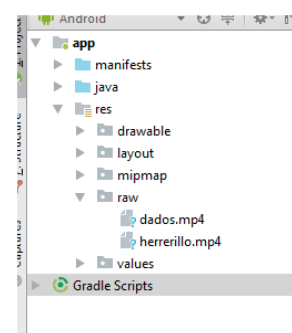
Figura 3



[Google Developers](#) (Uso educativo nc)

Los vídeos están sacados de Pixabay.com. Hay que tratar que no sean muy pesados para no sobrecargar la aplicación. Los alumnos podrán seleccionar los que quieran para practicar con este ejercicio.

Mostrar retroalimentación



[Google Developers](#) (Uso educativo nc)

En primer lugar habrá que incorporar los vídeos en la carpeta raw de la carpeta de recursos. La carpeta raw por defecto no está creada, pero podemos hacerlo mediante clic derecho sobre la carpeta res > New > Android Resource Directory. En el desplegable "Resource type" elegimos "raw", y ya le pondrá automáticamente también "raw" como "Directory name". Tras esto, copiamos los ficheros que en nuestro caso tendrán extensión **.mp4**. Quedará como se muestra en la imagen.

activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/a
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:padding="20dp"
    tools:context="com.example.dell.reproducirvideo.MainActivity"

    <Button
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:id="@+id/btnReproducir"
        android:text="Reproducir"
        android:layout_gravity="center"/>
```

```

17
18     <!-- Incorporamos etiqueta de View VideoView para cargar sobr
19     <VideoView
20         android:layout_width="match_parent"
21         android:layout_height="wrap_content"
22         android:id="@+id/vwReproductor"
23         android:layout_gravity="center"/>
24
25 </LinearLayout>

```

MainActivity.java

```

import android.net.Uri;
import androidx.appcompat.app.AppCompatActivity;
import android.support.design.widget.Snackbar;
import android.os.Bundle;
import android.view.View;
import android.widget.Button;
import android.widget.VideoView;

public class MainActivity extends AppCompatActivity {
    Button btnReproducir;
    VideoView vwReproductor;

    @Override

    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        btnReproducir=(Button)findViewById(R.id.btnReproducir);
        vwReproductor=(VideoView)findViewById(R.id.vwReproductor);
        btnReproducir.setOnClickListener(new View.OnClickListener()

        @Override

        public void onClick(View v) {
            int aleatorio=(int)Math.floor(Math.random()*2);
            int identificadorRecurso;
            if(aleatorio==0)
                identificadorRecurso = R.raw.dados;
            else
                identificadorRecurso = R.raw.herrerillo;
            String rutaVideo = "android.resource://com.exempl
            //En la línea anterior, en lugar de poner com.exa
            // con getPackageName(), la cual devuelve un Stri
            Uri uri = Uri.parse(rutaVideo);
            vwReproductor.setVideoURI(uri);
            vwReproductor.start();
        }
    }
}

```



2.2.- Ejercicio Resuelto 3.

Ejercicio Resuelto

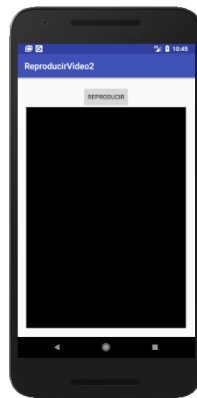
Incorporar en el ejercicio anterior la clase **MediaController**, la cual dotará al reproductor de un aspecto más completo, incorporando implícitamente controles de reproducir, parar, ir hacia delante e ir hacia atrás, e incluso arrastrar el cursor de la línea de reproducción para avanzar o adelantar todo lo que queramos la reproducción.

La actividad inicialmente tendrá el mismo aspecto (figura 1).

Además al clicar sobre el botón también se reproducirá aleatoriamente cualquiera de los vídeos (figura 2)

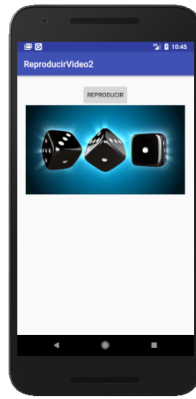
La diferencia es que si clicamos sobre la imagen del vídeo, ya me aparecen unos controles de reproducción proporcionados por la clase MediaController (figura 3).

Figura 1. Aspecto inicial



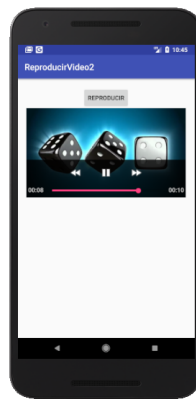
[Google Developers](https://developers.google.com/) (Uso educativo
nc)

Figura 2. Al clicar sobre el botón



[Google Developers](#) (Uso educativo
nc)

Figura 3. Al clicar sobre el vídeo



[Google Developers](#) (Uso educativo
nc)

Mostrar retroalimentación

Los recursos serán los mismos. Además el fichero **activity_main.xml** será exactamente igual. Los cambios se harán solo en **MainActivity.java**.

MainActivity.java

```
package com.example.dell.reproducirvideo2;
import android.net.Uri;
import androidx.appcompat.app.AppCompatActivity;
import android.os.Bundle;
import android.view.View;
import android.widget.Button;
import android.widget.MediaController;
import android.widget.VideoView;
```

```
//En este ejercicio, con respecto al anterior, incorporaremos la
//de un aspecto más completo, incorporando implícitamente control
//hacia atrás, e incluso podemos arrastrar el cursor de la línea
```

```
13 //que queramos la reproducción
14 public class MainActivity extends AppCompatActivity {
15     Button btnReproducir;
16     VideoView vwReproductor;
17     MediaController mediaController;
18
19     @Override
20
21     protected void onCreate(Bundle savedInstanceState) {
22         super.onCreate(savedInstanceState);
23         setContentView(R.layout.activity_main);
24         btnReproducir=(Button)findViewById(R.id.btnReproducir);
25         vwReproductor=(VideoView)findViewById(R.id.vwReproductor)
26         mediaController=new MediaController(this); //No poner get
27         btnReproducir.setOnClickListener(new View.OnClickListener
28
29             @Override
30
31             public void onClick(View v) {
32                 int aleatorio=(int)Math.floor(Math.random()*2);
33                 int identificadorRecurso;
34                 if(aleatorio==0)
35                     identificadorRecurso = R.raw.dados;
36                 else
37                     identificadorRecurso = R.raw.herrerillo;
38                 String rutaVideo = "android.resource://com.exempl
39                 Uri uri = Uri.parse(rutaVideo);
40                 vwReproductor.setVideoURI(uri);
41                 vwReproductor.setMediaController(mediaController)
42                 mediaController.setAnchorView(vwReproductor);
43                 vwReproductor.start();
44             }
45         });
46     }
47 }
```

2.3.- Ejercicio Resuelto 4.

Ejercicio Resuelto

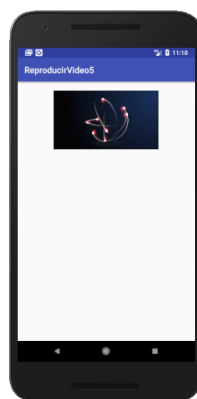
En este ejercicio, con respecto a los anteriores, vamos a proveer a nuestra aplicación de la capacidad de ser capaz de restaurar el estado de reproducción actual y posición, de forma que si cambiamos la orientación del dispositivo, en lugar de situarse al principio de la reproducción, sigue por donde estaba.

Esto viene a propósito de la nota mencionada anteriormente, en el capítulo explicativo sobre **VideoView**:

VideoView no conserva su estado completo al pasar al segundo plano o al cambiar la orientación del dispositivo. En particular, no restaura el estado de reproducción actual, posición de reproducción, pistas seleccionadas o cualquier pista de subtítulos agregada a través de **addSubtitleSource()**. Las aplicaciones deben guardarlas y restaurarlas por sí mismas en **Activity.onSaveInstanceState(Bundle)** y **Activity.onRestoreInstanceState(Bundle)**.

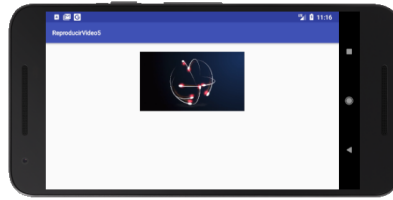
Dichos métodos son muy importantes porque harán que si giramos de orientación el dispositivo, la reproducción continuará por el punto donde estaban. Si no lo ponemos comenzará a reproducir desde el principio (se aconseja hacer la prueba).

Al reproducir el vídeo.



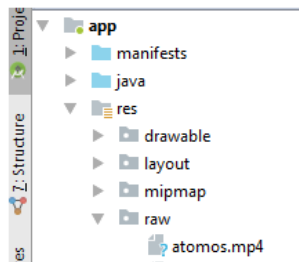
[Google Developers](https://developers.google.com/) (Uso educativo
nc)

Al girar la pantalla, se reanuda en el mismo punto en el que estaba cuando se giró.



[Google Developers](#) (Uso educativo nc)

Mostrar retroalimentación



Incorporaremos el vídeo (puedes elegir uno que tenga un peso razonable de algún banco de recursos) en la carpeta **raw**.

[Google Developers](#) (Uso educativo nc)

activity_main.xml

```

1  <?xml version="1.0" encoding="utf-8"?>
2  <LinearLayout xmlns:android="http://schemas.android.com/apk/res/a
3      xmlns:app="http://schemas.android.com/apk/res-auto"
4      xmlns:tools="http://schemas.android.com/tools"
5      android:layout_width="match_parent"
6      android:layout_height="match_parent"
7      android:orientation="vertical"
8      tools:context="com.example.dell.reproducirvideo5.MainActivity"
9
10     <VideoView
11         android:id="@+id/vv"
12         android:layout_width="wrap_content"
13         android:layout_height="wrap_content"
14         android:layout_margin="20dp"
15         android:layout_gravity="center" />
16
17 </LinearLayout>

```

MainActivity.java

```

package com.example.dell.reproducirvideo5;
import android.media.MediaPlayer;
import android.net.Uri;
import androidx.appcompat.app.AppCompatActivity;
import android.os.Bundle;
import android.util.Log;
import android.widget.MediaController;
import android.widget.VideoView;
/* En este ejercicio, con respecto a los anteriores, vamos a prov
de ser capaz de restaurar el estado de reproducción actual y posi
del dispositivo, en lugar de situarse al principio de la reproduc
*/

public class MainActivity extends AppCompatActivity {
    private VideoView vV;
    private int posicion = 0;

    @Override

    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        vV = (VideoView) findViewById(R.id.vV);
        establecerVideo();
    }

    private void establecerVideo() {
        // Especificamos la ruta al vídeo que reproduciremos
        //En mi caso getPackageName() devolverá la cadena "com.ex
        String ruta = "android.resource://" + getPackageName() +
        Uri uri = Uri.parse(ruta);
        // Para darle mayor funcionalidad, incorporaremos control
        MediaController mediaController = new MediaController(this)
        // Establecemos los controles multimedia a mi VideoView
        vV.setMediaController(mediaController);
        try {
            // Establecemos la URI del vídeo a mi VideoView
            vV.setVideoURI(uri);
            // Asignamos el foco a mi VideoView
            vV.requestFocus();
        } catch (Exception e) {
            Log.e("Error", e.getMessage());
            e.printStackTrace();
        }

        // Establecemos un escuchador de eventos o listener que c
        vV.setOnPreparedListener(new MediaPlayer.OnPreparedListen

        @Override

        public void onPrepared(MediaPlayer mp) {
            //Hacemos que el vídeo se reproduzca continuament
            mp.setLooping(true);
            if (posicion == 0) { //Si estamos en el principio

```

```
56         vV.start();
57     } else { //Si no está en principio, entonces lo p
58         vV.pause();
59     }
60 }
61 });
62 }
63 /* Ahora incorporaremos la sobrescritura de los métodos onSave
64    Dichos métodos son muy importantes porque harán que si gir
65    continuará por el punto donde estaban.
66    Si no lo ponemos comenzará a reproducir desde el principio
67    */
68
69 @Override
70
71 public void onSaveInstanceState(Bundle savedInstanceState) {
72     super.onSaveInstanceState(savedInstanceState);
73     // Con onSaveInstanceState almacenamos la posición por do
74     savedInstanceState.putInt("miPosicion", vV.getCurrentPosi
75     vV.pause();
76 }
77
78 @Override
79
80 public void onRestoreInstanceState(Bundle savedInstanceState)
81     super.onRestoreInstanceState(savedInstanceState);
82     // Con onRestoreInstanceState recuperamos la posición por
83     posicion = savedInstanceState.getInt("miPosicion");
84     vV.seekTo(posicion);
85 }
```

3.- WebView.

Caso práctico

- Acabo de hablar con nuestro cliente -informa Ada al equipo-. La empresa que ha solicitado la aplicación para las visitas guiadas, ha pedido que algunos de los vídeos no se reproduzcan directamente desde la app sino que la reproducción se haga desde una plataforma externa donde están actualmente alojados.

- ¿Eso nos perjudica? -pregunta Juan.

- ¡Todo lo contrario! -exclama Ada, quien les explica los beneficios de la petición del cliente.



[Mikhail \(Pexels\)](#)

La reproducción de los vídeos desde una plataforma externa otorga varias ventajas:

- ✓ Es posible actualizar los contenidos en la plataforma externa manteniendo los enlaces, y por tanto sin necesidad de actualizar la app.
- ✓ No es necesario guardar los medios en los dispositivos locales donde la app se está ejecutando.
- ✓ Por otro lado, permite contabilizar todas las reproducciones y aumentar los ratios en términos de visitas.

- Para conseguirlo -aclara Ada al equipo tras indicarles las ventajas de la petición del cliente-, vamos a implementar la clase **WebView** en la aplicación.

La clase **WebView** es una extensión de la clase **View** de Android que nos permitirá mostrar páginas web como parte del diseño de nuestra actividad, de manera perfectamente integrada. No incluye ninguna característica de un navegador web completamente desarrollado, como controles de navegación o una barra de direcciones. Todo lo que hace **WebView**, de forma predeterminada, es mostrar una página web.

Un escenario común en el que usar **WebView** es útil es cuando desea proporcionar información en su aplicación que podría necesitar actualizar, como un acuerdo de usuario final o una guía del usuario. Dentro de su aplicación Android, puede crear una Actividad que contenga una **WebView** y luego usarla para mostrar su documento alojado en línea.

Otro escenario en el que **WebView** puede ayudar es si nuestra aplicación proporciona datos a un usuario que requiera una conexión a Internet para recuperar datos, como el correo electrónico. En este caso, puede ser más fácil construir un **WebView** en la aplicación Android que muestra una página web con todos los datos del usuario, en lugar de realizar una solicitud de red, luego analizar los datos y representarlos en un diseño de Android. En su

lugar, podemos diseñar una página web que se adapte a los dispositivos Android y luego implementar un **WebView** en nuestra aplicación que cargue la página web.

3.1.- Añadiendo un WebView a nuestra aplicación.

Lo primero será añadir una etiqueta **<WebView>** en el layout donde queramos incorporar el **WebView**.

Código

```
1 <?xml version="1.0" encoding="utf-8"?>
2 <WebView xmlns:android="http://schemas.android.com/apk/res/android"
3     android:id="@+id/webview"
4     android:layout_width="fill_parent"
5     android:layout_height="fill_parent"
6 />
```

A continuación, para cargar la página en el **WebView**, utilizaremos el método **loadUrl()**.

Código

```
1 WebView myWebView = (WebView) findViewById(R.id.webview);
2 myWebView.loadUrl("http://www.example.com");
```

Además, previamente, para que la conexión se realice será necesario incorporar en el fichero **AndroidManifest.xml** el permiso de acceso a Internet (**android.permission.INTERNET**):

```
1 <manifest ... >
2     <uses-permission android:name="android.permission.INTERNET" />
3     ...
4 </manifest>
```

Igualmente, en caso de que vayamos a acceder a contenido no seguro (no https) es necesario añadir también este atributo a la etiqueta application en el archivo de manifiesto:

1 | `android:usesCleartextTraffic="true"`

3.2.- Habilitar JavaScript.

Por defecto la carga del JavaScript asociado a una página web está deshabilitado en un **WebView**. Para habilitarlo podremos hacerlo a través de la clase **WebSettings** mediante el método **setJavaScriptEnabled()**.

Código

```
1 | WebView myWebView = (WebView) findViewById(R.id.webview);  
2 | WebSettings webSettings = myWebView.getSettings();  
3 | webSettings.setJavaScriptEnabled(true);
```

Nota: En las versiones más modernas de API, además tenemos que incorporar las líneas:

```
1 | myWebView.setWebViewClient(new WebViewClient());  
2 | myWebView.setWebChromeClient(new WebChromeClient());
```